MATERIALIZE — Differentiable Inverse Design of 2D Mechanical Metamaterials

Complete reference for future users. This document explains the framework (theory), the code (architecture and API, with explanations), and how to use it (worked, runnable examples). It is self-contained; the per-folder `README.md` / `SOLVER_GUIDE.md` files remain the authoritative low-level references and are cross-linked where relevant.

Table of contents

---

1. What MATERIALIZE is

MATERIALIZE designs **2D mechanical metamaterials** — planar spring networks whose *effective* elastic behaviour (Poisson ratio ν, Young's modulus E, the full elastic tensor, and local/spatial patterns of them) is prescribed by the user and then realised automatically.

The workflow is **inverse design by gradient descent**:

```
target elastic response  ⟶  optimise per-bond stiffnesses k  ⟶  network that realises it
                              (gradients flow through a differentiable homogeniser)
```

Two things make this practical and are the heart of the project:

- **A differentiable forward solver** (Phase 2) that maps a spring network to its homogenised elastic tensor and is differentiable with respect to every bond stiffness, at any mesh size.
- **An inverse-design engine** (Phase 3) that turns "I want ν = −0.5 here, E stiff there, isotropic everywhere else" into a concrete set of bond stiffnesses, using that solver's gradients.

Every design is **independently verified**: after the optimiser produces a network, a *separate* NumPy simulation (periodic relaxation → virial/energy homogenisation) confirms the network actually behaves as

designed. The design path and the verification path are genuinely different codebases, so agreement is meaningful.

The forward model is the **"Disc-to-Continuum" (D2C)** homogenisation of Grossman & Boudaoud (PRR 2026, arXiv:2309.07844) — a *geometric / metric* formulation of discrete elasticity, described next.

---

2. The physical system

A design is a **2D triangulated spring network**:

- **Nodes** — points in the plane (a triangular lattice, optionally perturbed/disordered, periodic or open).
- **Triangles** — the Delaunay triangulation of the nodes; the elastic response is assembled per triangle.
- **Bonds (springs)** — the triangle edges. Each bond `b` has:
- a **stiffness** `k_b > 0` (the design variable), and
- a **rest length** `ℓ₀,b` (a second, optional design variable). A bond shared by two triangles has a single, consistent `k` (design variables are **per-bond**, not per-triangle-edge).

The spring energy of the network under a deformation is the usual `½ Σ_b k_b (ℓ_b − ℓ₀,b)²`. The question the forward solver answers is: **given the network, what is its homogenised (effective, continuum) elastic tensor?** — and the inverse engine asks the reverse.

Topologies used throughout (see `_common.make_topology`): `regular` (φ=ψ=1 triangular lattice), `aniso_str` / `aniso_shr` (anisotropic crystals), `disorder_lo` / `disorder_hi` (positional disorder η=0.20/0.35). Sizes are given by a half-width `N`; `N=14` ≈ 2.7k triangles, `N=42` ≈ 16k.

---

3. The framework: the geometric (D2C) homogenisation

This section is the conceptual core. It is what distinguishes MATERIALIZE from a classical spring-network/FEM optimiser, and understanding it explains both the power and the honest limits of the method.

3.1 Metric, not displacement

Classical discrete elasticity is a **displacement** theory: the unknown is the nodal displacement field `u`; you assemble a global stiffness matrix `K(k)`, solve the equilibrium `K u = f` for where the nodes go, and *measure* the modulus from the solved field.

D2C is a **metric** theory. The primary object is the discrete **metric change** per triangle,

```
Δg(s) = F(s)ᵀ F(s) − I          (Voigt vec3 = [g_xx, g_xy, g_yy])
```

— the intrinsic geometry of each triangle — and **node displacements never appear**. Each triangle's "bare" elastic tensor `A(s)` is a *closed-form* function of its geometry and bond stiffnesses (no solve needed to get the material). The homogenisation is a **compatibility** problem, not an equilibrium solve: find the

per-triangle metric-fluctuation field `δg(s)` that minimises the spring energy subject to the fluctuations being *geometrically realisable*.

One line captures the distinction:

- **Classical:** strain `= sym(∇u)` (compatible by construction), solve **equilibrium** for `u`.
- **D2C:** the metric strain is *primary*; impose **compatibility** explicitly, solve for the metric response — never embed, never compute a displacement.

3.2 Linear in energy, exact in geometry

A crucial and often-misread point:

> The D2C formulation is **linear in ENERGY, not in GEOMETRY.**

The strain measure `Δg = FᵀF − I` is the full, geometrically **exact** metric change (it includes the quadratic term in the displacement gradient) — *not* the linearised engineering strain `sym(∇u)` of linear elasticity. The only linearisation is the **constitutive law**: the elastic energy is a *quadratic* form in that exact metric strain (a linear stress–strain relation).

Placement versus FEM:

method	kinematics (geometry)	constitutive (energy)
linear FEM	linearised (`sym∇u`)	quadratic
nonlinear FEM	exact	can be nonlinear
D2C (this)	**exact (`FᵀF−I`)**	**quadratic (linear)**

So D2C is geometrically as honest as nonlinear FEM at linear-FEM material cost. A practical consequence: the per-triangle **strain-concentration operator** `W(s)` that the solver returns is a *linear-response* operator (it is linear because the energy is quadratic), but the kinematics behind it are exact. Do **not** describe `W` as "the geometry being linearised."

3.3 One solve gives the whole response operator ("direct")

The solver does **not** answer "what happens under this one load." It solves, once, for the strain-concentration field `W(s)` — the *complete* linear map from any macroscopic strain to each triangle's local metric response. From that single object fall, with no further solves:

- the homogenised effective tensor `C_eff` (all independent components at once),
- the directional response ν(θ) and E(θ) at **every** angle,
- the per-triangle strain and stress under **any** applied macroscopic load.

This is why the framework is best described as **direct** rather than merely "fast": the material is a closed-form read-off per triangle, the full response operator is the primary output, and its gradient with respect to the design variables is immediate (no adjoint of a global displacement solve). It is *also* why targeting

local/per-element strain or stress (Section 6) is natural here and awkward in a displacement code — the operator you need is already the output.

Because ν and E are both contractions of the *same* `C_eff`, they are **coupled**: you cannot prescribe them fully independently (see realizability in Section 6).

3.4 The compatibility constraints (why it is correct)

The metric-fluctuation solve minimises `½ Σ_s (Δḡ + δg_s)ᵀ H_s (Δḡ + δg_s)` over the per-triangle field `δg`, subject to three **intrinsic** constraints:

1. **edge agreement** — shared edge lengths match between neighbouring triangles;
2. **zero discrete Gaussian curvature** (`inc(δg)=0`) — the vertex angle-deficit / St-Venant compatibility condition, so the fluctuation field is realisable as an actual (flat) deformed sheet;
3. **area-weighted normalisation** `Σ_s A_s δg_s = 0`.

The historically-important subtlety (`THEORY_NOTES.md`): the *plain* mean `Σ_s δg_s = 0` — used by the older single-site mean field — is **wrong**. It over-constrains the realisable fields and produces a systematic ≈1.4× over-compliance. The correct normalisation is **area-weighted**, which is a discrete divergence-theorem identity (`Σ_s A_s ε_s ≡ 0` for any periodic displacement field) — it is *implied by compatibility* and therefore harmless, whereas the plain mean injects a constraint inconsistent with realisable fields. With the curvature constraint + area-weighted normalisation, the intrinsic metric solve reproduces the periodic simulation essentially exactly (correlation ≈1.0, overshoot 1.000 at all disorder levels), to first order in the strain amplitude.

3.5 Homogenisation is the unweighted mean

The final effective tensor is the **unweighted** per-triangle mean of `C(s) = (I+W)ᵀ A(s) (I+W)` — this is the true physical (energy = virial) modulus. (An area weight *is* used in the compatibility normalisation above, where it is correct; it is *not* used in the final average, where it would bias ν on disordered/anisotropic meshes.) Verified against the virial and energy-Hessian ground truths (`verification_tools/physical_homog.py`).

3.6 Honest limits

- **Linear constitutive law** — no material nonlinearity (strain-stiffening, buckling of the constitutive response). The tensor `C` is the tangent about the given reference (geometry + ℓ_o).
- **Near a mechanism it is fragile.** When a design sits near a soft-mode / mechanism threshold, the quadratic energy has a near-zero eigenvalue and the linear response operator `W` becomes ill-conditioned; the linear readback then diverges from the true nonlinear relaxation. This is a soft-eigenvalue issue present for *any* method at a mechanism — it is **not** a geometric linearisation artifact — and it is precisely why every design is checked against the independent nonlinear simulation (Section 8). Several demos (the concentric-ring "bullseye" at high contrast, strong strain "bulges") hit this limit deliberately and document it.
- **Periodic / bulk by construction.** D2C computes the homogenised bulk response. Genuinely open-boundary specimen questions (a cut, free-edge stretch test) are answered by a separate classical nodal spring solve used only for *verification* (`_common.open_stretch`), never in the design loop.

---

4. Repository layout

```
MATERIALIZE/
├── Phase 2/                    the differentiable FORWARD solver (protected core)
│   ├── forward_solver_torch.py  ElasticSolver: network -> C_eff, nu, E, W  (differentiable)
│   ├── SOLVER_GUIDE.md          canonical usage/API guide for the solver
│   └── test_forward_solver.py   regression tests (nu=1/3 crystal, gradients, adjoint, ...)
├── Phase 3/                    INVERSE design
│   ├── inverse_design.py        DesignProblem, Objective, optimize, validate, constrain
│   ├── test_inverse_design.py   16-test suite (round-trip, auxetic, local, mixed, strain/stress,
│   │                            homogenisation, isotropisation, ...)
│   ├── README.md                inverse-design concepts + examples
│   └── verifications/           end-to-end design→simulate→check demos (see Section 8)
│       ├── _common.py           shared harness (topologies, plotting, independent simulation)
│       └── <case>/              auxetic_sweep, auxetic_patch, anisotropy, two_region,
│                                dir_aux_ribbon, strain_stress, large16k, ...
├── Phase 4/                    ML surrogates & generative design (GNN/VAE) — see Tutorial.md
├── verification_tools/         physical ground truth (physical_homog.py) + analysis scripts
├── THEORY_NOTES.md             why the intrinsic metric solve is correct (the derivation notes)
├── INTRINSIC_METRIC_SOLVE.md   the intrinsic KKT formulation in detail
└── documentation/              THIS document (.md + .pdf)
```

Protected core: `Phase 2/forward_solver_torch.py` is gated by `test_forward_solver.py` and the physical `verify_*` suite; it is not modified without an explicit request and a passing regression.

---

5. Phase 2 — the differentiable forward solver

Reference: `Phase 2/SOLVER_GUIDE.md`.

5.1 What it computes

Given a triangulation + per-triangle-edge stiffness `k` (shape `(N,3)`) and rest length ℓ_o, one `forward(...)` call returns the homogenised effective tensor and, from it, ν and E — **plus** the per-triangle bare tensor `bare` and strain-concentration `W`. Fully differentiable w.r.t. `k` and ℓ_o.

Pipeline (intrinsic metric solve — the verified default; no node displacements):

1. per-triangle metric Hessian $A(s) = \Sigma_e (k_e / 4\ell_e^2)\, q_e q_e^T$;
2. solve the constrained saddle for $W(s)$ (energy minimised s.t. edge compatibility + curvature + area-weighted mean, Section 3.4);
3. per-triangle actual tensor $C(s) = (I+W)^T A(s) (I+W)$; homogenise (unweighted mean) → $C_{eff} \to \nu, E$.

5.2 The `forward` call

```
out = solver.forward(rigidities, rest_lengths=l0,
                     method='intrinsic',      # default (verified); 'woodbury' = legacy mean field
                     physical_units=False)    # True -> E / C in true physical units
# convenience: solver(k, l0) == solver.forward(k, l0)
```

Returns a dict:

key	shape	meaning
`poisson`	scalar	Poisson ratio ν (physical, scale-invariant — correct either way)
`young`	scalar	Young's modulus E (internal scale; physical if `physical_units=True`)
`elastic_tensor`	`(6,)`	homogenised `C_eff` (6 independent components)
`per_triangle`	`(N,6)`	per-triangle actual tensor `C(s)` — **use for local/regional objectives**
`bare`	`(N,5)`	per-triangle bare tensor `A` (differentiable)
`W`	`(N,9)`	per-triangle strain-concentration `∂δg_loc/∂Δḡ` (differentiable)

`bare` and `W` are the operators behind the per-triangle strain/stress objectives (Section 6.3).

5.3 Units

- **ν is always physical** (dimensionless / scale-invariant).
- **E and `elastic_tensor`** are in an internal scale by default; pass `physical_units=True` to rescale to true physical units (the unit triangular lattice → `E = 2/√3 ≈ 1.155`). Exact for periodic meshes; slightly approximate on open finite samples (ν unaffected).

5.4 Differentiability at any mesh size (the adjoint)

The intrinsic solve chooses its implementation automatically by triangle count (`INTRINSIC_DENSE_MAX = 600`):

N_tri	path	gradients
≤ 600	dense torch Woodbury	✅ autograd
> 600, no grad	sparse scipy `splu`	forward-only
> 600, **grad**	**adjoint** (`_IntrinsicSparseWFn`)	✅ autograd

The large-mesh adjoint reuses the forward's sparse factorisation (the saddle is symmetric), so gradients add only ~1.07× over forward-only and scale as sparse `O(N^1.5)` — gradient-based design works into the thousands of triangles (this project routinely designs at ~16k). It triggers automatically when the input requires grad.

---

6. Phase 3 — inverse design

Reference: `Phase 3/README.md`.

6.1 The three objects

`DesignProblem` wraps a network + solver + the per-bond→per-triangle map. Design variables are per-bond.

```
prob = DesignProblem.periodic(N=14, eta=0.3, seed=0)   # periodic perturbed-lattice unit cell
prob = DesignProblem.open(tri)                          # open mesh from a scipy triangulation
```

`Objective(kind, target, region=None, weight=1.0, thetas=None, load=None, homogeneity=0.0)` — one target over a region (`region=None` ⇒ global; an array of triangle indices ⇒ local).

kind	meaning
`'nu'` / `'E'`	a **scalar = ISOTROPIC** target: that value in *every* direction (ν(θ)/E(θ) flat). This is the default meaning of "ν = ν".
`'nu_dir'` / `'E_dir'`	legacy single-direction scalar (pins one orientation only; tensor may stay anisotropic).
`'tensor'`	the full physical 6-vector.
`'nu_theta'` / `'E_theta'`	a directional profile over `thetas` (default `ANG=linspace(0,π,37)`); scalar broadcasts to flat.
`'isotropy'`	penalise the anisotropic part of the tensor (level free).
`'strain'` / `'stress'`	the **actual per-triangle metric-change response** (vec3=[xx,xy,yy]) under an explicit applied macro `load`.

Extra arguments: - `load` — the applied macroscopic Δḡ (vec3), **required** for `'strain'` / `'stress'`. - `homogeneity` — `>0` adds a penalty on the *variance* of the local response within the region (Section 6.4).

`optimize(prob, objectives, mode='k', optimizer='lbfgs', n_iter, n_restarts, reg, seed)` — runs the design. `mode ∈ {'k','l0','both'}`. Returns `dict(k, l0, loss, history)`. Variables are softplus-positive (`k = softplus(raw)`); the loss is a weighted sum of per-objective distances, minimised with L-BFGS (strong-Wolfe). Because `k→C_eff` is `~3·N_bond → 6`, the problem is heavily underdetermined, so L-BFGS hits targets in tens of iterations. `n_restarts>1` guards against the occasional unlucky local optimum (L-BFGS run-to-run nondeterminism). `reg` is a small uniformity regulariser that discourages exploiting floppy/unstable configurations.

`validate(prob, k, l0, objectives)` — re-evaluates each objective at the designed params and returns achieved-vs-target (self-consistency of the solver's own forward pass). For an *independent* check, simulate the designed network (Section 8).

6.2 `constrain(...)` — fix some quantities, free the rest

```
constrain(region=R, nu=ν)                       # isotropic nu=ν, E free
constrain(region=R, E=v)                         # isotropic E=v, nu free
constrain(region=R, isotropic=True)              # direction-independent, level(s) free
constrain(region=R, tensor=isotropic_c6(nu, E))  # EXACT isotropic (nu AND E), nothing free
```

Why "scalar = isotropic" matters: on a regular lattice with a central auxetic patch (ν(θ) measured *inside*), `Objective('nu_dir', -0.3)` gives ν(θ) range **1.69** (wildly anisotropic), whereas `Objective('nu', -0.3)` gives range **0.011** and `constrain(tensor=isotropic_c6(-0.3,·))` → **0.001**.

6.3 Strain / stress objectives — the "full response operator" in action

Because the solver returns `bare` and `W`, the *actual* per-triangle strain and stress under any applied macroscopic load are one differentiable contraction (`per_triangle_strain_stress(bare, W, load)`). This enables objectives on the raw local response, not just derived ν/E:

- `Objective('stress', target=[σxx,σxy,σyy], region=R, load=Δğ)` — target a region's mean stress under `load`.
- `Objective('strain', target=[εxx,εxy,εyy], region=R, load=Δğ)` — target a region's mean strain. (Whole-cell mean strain is degenerate — it always equals the applied load — so `'strain'` is only meaningful on a sub-region; `'stress'` is designable everywhere.)

The convention (Voigt basis, the `(I+W)@load` strain-concentration formula, no extra factor-of-2 on shear) is pinned *empirically* against the independent NumPy simulation (`test_inverse_design.py` test [10]).

6.4 The homogeneity regulariser

`Objective(..., homogeneity=w)` adds `w · var(local_field[region])` to the loss for any `nu` / `E` / `strain` / `stress` objective. It discourages the optimiser from hitting a region *mean* target via a few floppy (k→0) hinge triangles while the rest of the region is untouched — i.e. it prefers a *uniform* local response. This is the loss-level analogue of the geometry-level `glue()` technique used for multi-region designs (see `verifications/README.md`).

---

7. Worked examples

All examples assume `from inverse_design import DesignProblem, Objective, optimize, validate` and run from `Phase 3/`.

7.1 Forward: compute ν and E of a network

```
import torch
prob = DesignProblem.periodic(N=14, eta=0.0, seed=0)          # regular lattice
out = prob.forward(torch.ones(prob.n_bond), physical_units=True)
print(out['poisson'], out['young'])                           # ~0.333, ~1.155 for the unit crystal
```

7.2 Global auxetic target

```
prob = DesignProblem.periodic(N=14, eta=0.3, seed=0)
res = optimize(prob, [Objective('nu', target=-0.2)], mode='k', n_iter=80)
print(validate(prob, res['k'], res['l0'], [Objective('nu', -0.2)]))   # achieved nu ~ -0.200
```

7.3 Match a full elastic tensor (round-trip)

```
import torch
k_true = 0.3 + 1.5 * torch.rand(prob.n_bond)
target = prob.region_tensor(prob.forward(k_true)['per_triangle'], None).detach()   # (6,) physical
res = optimize(prob, [Objective('tensor', target)], mode='k', n_iter=120)          # err ~ 1e-5
```

7.4 Local patch — a specific response in a sub-region

```
prob = DesignProblem.periodic(N=16, eta=0.2, seed=4)
patch = prob.region_in_circle(prob.centroids.mean(0), radius=1.5)
res = optimize(prob, [Objective('nu', target=-0.15, region=patch)], mode='k', n_iter=120)
```

7.5 Mixed — global average with a local auxetic patch (the headline capability)

```
prob = DesignProblem.periodic(N=18, eta=0.2, seed=5)
patch = prob.region_in_circle(prob.centroids.mean(0), radius=1.4)
objs = [Objective('nu', target=+0.25, region=None,  weight=1.0),   # whole network stays positive
        Objective('nu', target=-0.20, region=patch, weight=2.0)]   # the patch is auxetic
res = optimize(prob, objs, mode='k', n_iter=150)
# validate -> global nu ~ +0.25 AND patch nu ~ -0.20, simultaneously
```

7.6 Directional response ν(θ) / E(θ)

```
import numpy as np
from inverse_design import ANG
prob = DesignProblem.periodic(N=20, eta=0.3, seed=0)
# program a 2-fold Poisson profile:
optimize(prob, [Objective('nu_theta', 0.2 + 0.35*np.cos(2*ANG))], mode='k', n_iter=110)
# isotropise an anisotropic base to a chosen flat level (scalar broadcasts):
optimize(prob, [Objective('nu_theta', -0.2)], mode='k', n_iter=150)
# independent anisotropy: flat nu, directional E:
optimize(prob, [Objective('nu_theta', 0.2,                        weight=4.0),
                Objective('E_theta',  1.0*(1+0.4*np.cos(2*ANG)),   weight=1.0)], mode='k', n_iter=150)
```

7.7 Per-triangle stress / strain under a load

```
import numpy as np, torch
LOAD_X = np.array([1.0, 0.0, 0.0])          # a pure x-stretch (macro Delta_g vec3)
patch  = prob.region_in_circle(prob.centroids.mean(0), radius=1.5)
# design a patch to carry an amplified sigma_xx under x-pull:
optimize(prob, [Objective('stress', target=torch.tensor([0.35,0,0]), region=patch, load=LOAD_X)],
         mode='k', n_iter=150)
```

7.8 Large network (≈16k triangles) — gradients via the adjoint, no code change

```
prob = DesignProblem.periodic(N=42, eta=0.35, seed=0)    # ~16k triangles
res = optimize(prob, [Objective('nu', target=-0.4)], mode='k', n_iter=70)
```

---

8. The verification suite

Reference: `Phase 3/verifications/README.md`. Every case **designs** k, then **independently simulates** the designed network (a separate NumPy periodic relaxation → physical per-triangle tensor via `verification_tools/physical_homog.py`) and checks it does as prescribed. The design path (differentiable `forward()`) and the verification path (non-autograd virial/energy homogenisation) are genuinely different code.

Headline cases (each saves a CSV of numbers + figures, and its designed networks as `.npz`):

- `auxetic_sweep` — global ν over a sweep; establishes the topology-dependent auxetic limit (disordered η=0.35 reaches ν=−0.6; the regular crystal ≈ −0.3).
- `auxetic_patch` — local/spatial control: auxetic discs/squares/triangles/rings in a normal matrix, decoupled E-region + ν-region, etc.
- `anisotropy` — program any ν(θ)/E(θ), isotropise a base, or make E directional while ν stays flat.
- `two_region` — does a design survive a **real open cut-and-stretch**, and does gluing two differently-behaved regions produce a differential response? Introduces `_common.glue()` (design each region independently, then retriangulate the union and transfer stiffnesses) — the robust way to build multi-region materials, since jointly optimising one connected lattice drives interface bonds to k=0 (a hinge/mechanism).
- `strain_stress` — the per-triangle strain/stress objectives in action:
- *stress concentrator / strain shield* — a patch that carries amplified σ_xx / near-zero strain.
- *auxetic-inclusion bulge* — a ν<0 patch in a ν=0 matrix that bulges laterally under a real pull.
- *concentric rings ("bullseye")* — alternating stress bands; the achievable version (16k, same-sign alternating **magnitude**) is clean, while alternating **sign** is a documented negative result (thermodynamically forbidden for a passive sub-region under global dilation).
- *mode-selective* — a **single** network whose pattern depends on the load direction: one shape responds to εxx, another to εyy, designed in **one** `optimize()` call by querying the same `W` at two loads. The *strain* version renders clean filled shapes (strain localises in soft inclusions); the *stress*

version is direction-selective but the shapes follow load paths (a documented physical distinction — see Section 3.6 and the case comments).

Two honest lessons, written into the demos: (i) *stress follows equilibrium load paths* — you cannot confine high stress to an isolated blob under a uniaxial load; load-aligned shapes render cleanly, shapes across the load grow a feeder streak. (ii) *strong local response ⇒ soft material* — strongly dilating/auxetic inclusions must be near-mechanism soft (E→0), which is exactly the fragile regime where the linear readback needs the independent nonlinear check to validate it.

Run the regressions:

```
python "Phase 2/test_forward_solver.py"     # solver: nu=1/3 crystal, gradients, adjoint, ...
python "Phase 3/test_inverse_design.py"     # 16 tests: round-trip, auxetic, local, mixed,
                                            # strain/stress, homogenisation, isotropisation, ...
```

---

9. Conventions, units, and caveats

- **Strain convention.** The framework's native "strain" is the metric change `Δg = FᵀF − I`, in the Voigt basis `vec3 = [xx, xy, yy]` with **no factor-of-2 on shear**. This is *not* the linearised engineering strain. Local strain under a load is `(I + W) @ load`; local stress is `A : strain` (`bare_stress`).
- **ν vs E are coupled** — both are contractions of the one `C_eff`, so they cannot be prescribed independently in general; a `tensor` / `isotropic_c6` target pins the whole thing.
- **Units** — ν is always physical; pass `physical_units=True` for physical E (Section 5.3).
- **`ℓ₀` degeneracy** — the rest length is a differentiable input but currently enters *only* as `k/ℓ₀²`, so designing `ℓ₀` at fixed geometry is mathematically degenerate with designing `k`. True reference-metric / residual-stress physics (incompatible `ℓ₀`) is not yet implemented; the variable is exposed but degenerate.
- **Near-mechanism fragility** — see Section 3.6. `min(k)==0` alone is *not* a reliable mechanism indicator (softplus underflows for very negative raw-k, common even in good designs); the honest metric is the per-triangle response *variance* within a region (what the `homogeneity` penalty controls) and, ultimately, the independent nonlinear simulation.
- **Protected core** — do not modify `Phase 2/forward_solver_torch.py` without a passing `test_forward_solver.py` and the physical `verify_*` suite.
- **Do not** compare against the legacy metric average (`test_cluster_Ceff.Ceff_nuE`) — it is the superseded convention; use the physical (virial/energy) ground truth.

---

10. Quick API reference

Forward solver (`Phase 2/forward_solver_torch.py`)

```
out = solver.forward(k, rest_lengths=l0, method='intrinsic', physical_units=False)
#   -> dict(poisson, young, elastic_tensor(6), per_triangle(N,6), bare(N,5), W(N,9))
```

Inverse design (`Phase 3/inverse_design.py`)

```
prob = DesignProblem.periodic(N, eta, seed)            # or .open(tri) / .from_geo(geo)
prob.region_in_circle(center, radius)                  # -> triangle-index array (a region)
prob.region_tensor(per_triangle, region)               # region-mean physical 6-vector
prob.forward(k, l0=None, physical_units=True)          # thin wrapper over the solver

Objective(kind, target, region=None, weight=1.0, thetas=None, load=None, homogeneity=0.0)
#   kind in {'nu','E','nu_dir','E_dir','tensor','nu_theta','E_theta','isotropy','strain','stress'}

constrain(region=None, weight=1.0, *, nu=None, E=None, isotropic=False, tensor=None,
          nu_theta=None, E_theta=None, thetas=None, nu_scalar=None, E_scalar=None)
isotropic_c6(nu, E)                                     # the isotropic 2D physical 6-vector

optimize(prob, objectives, mode='k', optimizer='lbfgs', n_iter=80, n_restarts=1, reg=1e-4, seed=0)
#   -> dict(k, l0, loss, history)
validate(prob, k, l0, objectives)                      # achieved-vs-target per objective

per_triangle_strain_stress(bare, W, load)              # (I+W)@load and A:strain, differentiable
region_mean_vec3(field, region)                        # region-mean of an (N,3) field
```

Independent verification (`Phase 3/verifications/_common.py` , `verification_tools/physical_homog.py`)

```
C.make_case(topo, N)               # (DesignProblem, geo)
C.sim_per_triangle_C6(geo)         # per-triangle physical tensor from ONE PBC relaxation (sim)
C.region_phys_C6(geo, C6, region)  # region-mean physical 6-vector (independent ground truth)
C.unit_mode_response(geo)          # per-triangle strain/stress under the 3 unit modes (sim)
C.local_nuE_angleavg(geo, C6)      # per-triangle angle-averaged (nu, E) material maps
C.open_stretch(geo, axis=0)        # real open-boundary cut-and-stretch (classical nodal solve)
```

For the theory in full, see `THEORY_NOTES.md` and `INTRINSIC_METRIC_SOLVE.md`. For the ML surrogate/generative layer (Phase 4), see `Tutorial.md`. For the code-review/refactor history of the verification harness, see `CODE_REVIEW.md`.